1. Что такое telegram-bot и каковы его основные возможности? Функциональные возможности telegram-bot и примеры его использования

Telegram-bot — это аккаунт Telegram, управляемые программой через Telegram Bot API. Он обрабатывает команды, отправляет сообщения, Телеграм-боты могут работать круглосуточно, мгновенно отвечая на запросы и автоматизируя рутинные задачи, такие как рассылка уведомлений или модерация чатов. Также боты могут проводить опросы, собирать данные клиентов и обрабатывать платеже.
Примеры: уведомление о заказах, мониторинг цен, запись на услуги, получение новостей.

2. Как создать простейший Telegram-бот с использованием библиотеки python-telegram-bot? Расскажите последовательность действий от регистрации бота в BotFather до написания минимального кода.

- Регистрация в BotFather: в Telegram открыть @BotFather → /newbot → → задать имя и username → получить токен.
- Установка библиотеки: pip install python-telegram-bot.
- Минимальный код:

```
from telegram import Update
from telegram.ext import ApplicationBuilder, CommandHandler, ContextTypes
```

Функция-обработчик команды

```
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Привет!")
def main():
    app = ApplicationBuilder().token("TOKEN").build()
    app.add_handler(CommandHandler("start", start))
    app.run_polling()
if __name__ == "__main__":
    main()
```

Регистрация обработчика команды

3. Как обработать команду /start в Telegram-боте? Опишите механизм регистрации команд и вызова соответствующих обработчиков.

- Обработка /start:
  - на python-telegram-bot — вопрос 2.
  - на aiogram:

```
from aiogram import Bot, Dispatcher, executor, types
API_TOKEN = "token"
bot = Bot(token = API_TOKEN)
dp = Dispatcher(bot)
@dp.message_handler(commands=["start"])
async def start(message: types.Message):
    await message.answer("Привет")
if __name__ == "__main__":
    executor.start_polling(dp, skip_updates=True)
```

Регистрация обработчика команды

4. Какие типы обновлений (updates) предоставляет Telegram Bot API? Перечислите основные типы и опишите их назначение.

Update — это контейнер для любого нового события, которое бот получает от Telegram.

Основные типы:
- message — новое сообщение
- edited_message — сообщение, которое было изменено пользователем.
- channel_post — новый пост в канале, где бот является админом.
- edited_channel_post — изменённый пост в канале
- inline_query — запрос от пользователя через inline-режим (когда бот вызывается прямо в строке ввода, например @bot_name)
- chosen_inline_result — результат, который пользователь выбрал из предложенных inline-ответов
- callback_query — ответ от кнопки inline-клавиатуры (кнопки под сообщением)
- poll — новое голосование, созданное ботом или пользователем
- poll_answer — ответ на голосование
- chat_member — изменение статуса участника чата (вступление, покидание)
- my_chat_member — изменение статуса бота в чате
- chat_join_request — запрос на вступление в чат/группу, где бот админ

5. Как реализовать inline-кнопки в сообщениях Telegram-бота? Что такое inline-кл-ра и как с ней взаимодействовать

Inline-клавиатура в Telegram Bot — это набор кнопок, прикрепленных непосредственно к сообщению, которые не заменяют основную клавиатуру пользователя, а появляются под текстом. При нажатии они генерируют callback_query без отправки нового сообщения.

Реализация на python-telegram-bot:

```
from telegram import InlineKeyboardButton,
                     InlineKeyboardMarkup
async def start (update, context):
    kb = InlineKeyboardMarkup([ — создание клавиатуры
        [InlineKeyboardButton("Кнопка", callback_data = "button_pressed")]
    ])
    await update.message.reply_text("Выберите действие:", reply_markup=kb) — отправка сообщения с клавиатурой
async def button_hadler (update, context): — обработчик нажатия кнопки
    query = update.callback_query
    query.answer() - подтверждаем нажатие
    query.edit_message_text (text = "Вы нажали кнопку")
async def main():
    updater = Updater ("токен")
    dp = updater.dispatcher
    dp.add_handler (CommandHandler("start", start))
    dp.add_handler (CallbackQueryHandler (button_handler)) — обработка кнопок
    updater.start_polling()
if __name__ == "__main__":
    main()
```

6. Что такое Callback Query и как его обрабатывать? В каких случаях возникает Callback Query и примеры его обработки.

Callback Query — объект обновления (Update), который Telegram отправляет боту, когда пользователь нажимает inline-кнопку с параметрами: id, message, from, callback_data.

Обработка делается через: • Callback Query Handler (в python-telegram-bot) - вопрос 5
• @dp.callback_query_handler (в aiogram)

```
from aiogram import Bot, Dispatcher, executor, types
API_TOKEN = "token"
bot = Bot(token = API_TOKEN)
dp = Dispatcher(bot)
@dp.message_handler(commands =["start"])
async def start (message:types.Message):
    keyboard = types.InlineKeyboardMarkup()
    keyboard.add(types.InlineKeyboardButton("Кнопка", callback_data = "btn1"))
    await message.answer("Выберите действие:", reply_markup = keyboard)
@dp.callback_query_handler(lambda c: c.data == "btn1") - ф-ция, проверяющая поле data. если = btn1, то run.
async def process_callback (callback_query: types.CallbackQuery):
    await callback_query.answer("Кнопка нажата!")
    callback_query.message.edit_text("Вы нажали кнопку!")
if __name__ == "__main__":
    executor.start_polling(dp, skip_updates = True)
```

7. Каким образом можно реализовать многофункциональные команды в Telegram-боте? Объясните особенности обработки разных команд и аргументов

Многофункциональные команды реализуются через передачу аргументов. Разные команды регистрируются отдельными обработчиками. Аргументы позволяют делать команды многофункциональными.

```
@dp.message_handler(commands =["manage"])
async def manage (message: types.Message):
    args = message.get_args().split() - получаем аргументы после команды
    if not args:
        await message.answer("Вы не передали аргументы")
        return
    action = args[0]
    if action == "add" and len(args) >1:
        await message.answer(f"Добавлен user: {args[1]}")
    else:
        await message.answer("Неизвестное действие.")
```

8. Как организовать хранение данных о пользователях или сессиях в Telegram-боте? Подходы от in-memory хранилищ до использования баз данных.

Способы хранения:

1) In-Memory: · использует структуры данных python (dict, list, set)
· данные живут только пока работает бот

2) JSON, CSV, TXT: · сохранение данных в локальные файлы
· данные не зависят от работы бота

3) SQLite: · легкая встроенная БД (один файл)
· подходит для небольших проектов

4) PostgreSQL, MySQL, MongoDB): · поддержка сложных запросов, масштабируемости
· используется для крупных проектов

5) Redis: · удобно для сессий
· in memory база для временных данных

9. Что такое объект Context в библиотеке python-telegram-bot и как его использовать. Преимущества передачи контекста между обработчиками

Context – это объект, передаваемый во всех обработчики вместе с Update. Он содержит состояние чата/пользователя, аргументы команд, служебные данные для передачи между обработчиками. С его помощью можно удобно сохранять данные о пользователях и чатах без внешней базы данных.

Преимущества: · Упрощение логики (Вместо того, чтобы каждый обработчик заново искал данные, они передаются через конспект.
· Проще подключать новые обработчики
· Данные централизированы в контексте, что дает возможность всем обработчикам работать с актуальной информацией

Пример:

```
.....
from aiogram.dispatcher import FSMContext
from aiogram.dispatcher.filters.state import State, StatesGroup
class Registration(StatesGroup):  - Определяем класс состояний для регистр.
    name = State()
    age = State()
@dp.message_handler(commands=['start'])
async def start(message: Message):
    await message.answer("Введите ваше имя:")
    Registration.name.set()  - переводим в состояние "name"
@dp.message_handler(state = Registration.name)
async def process_name(message: Message, state: FSMContext):
    await state.update_data(name = message.text)  - сохраняем введенное имя в контексте
    await message.answer("Регистрация завершена")
    state.finish()  - завершаем состояние
```

10. Как настроить вебхук для Telegram-бота? Отличия между опросом (polling) и вебхуками, а также порядок настройки вебхука

Вебхук – способ, при котором один сервер автоматически отправляет данные другому, как только происходит событие.

Отличия:

метод	описание	скорость	нагрузка на сервер	настройка
webhook	Telegram сам отправляет обновления на указанный URL вашего сервера	Почти мгновенная доставка	Низкая, т.к. не постоянных запросов.	Требуется публичный HTTPS-сервер
polling	Бот сам опрашивает сервер Telegram на наличие новых сообщений	Может быть задержка между запросами	Высокая из-за постоянных запросов	Простая, можно запускать локально

Порядок настройки:

Требования:
- публичный домен
- https с валидным SSL-сертификатом
- сервер, принимающий POST-запросы
- токен бота

1. Поднять https endpoint – сервер, который принимает POST-запросы от tg.
пример на fastAPI:

```
from fastapi import FastAPI, Request
app = FastAPI()
@app.post("/telegram")
async def telegram_webhook(req: Request):
    data = await req.json()
    print(data)
    return {"ok": True}
```

2. Указать Telegram URL вебхука:
- через запрос https://api.telegram.org/bot<TOKEN>/setWebhook?url=https://example.com/telegram

3. Принимать на endpoint JSON update и передавать его в обработку.

11. Какие методы существуют для отправки мультимедийного контента ботом? Методы API и особенности их использования

Тип контента	Метод API	Особенности использования
Фото	sendPhoto	Поддерживает URL, file_id или загрузку файла. Можно добавить подпись и кнопки
Аудио	sendAudio	Поддерживает MP3/OGG. Можно указать исполнителя, название, длину
Голос	sendVoice	Для коротких голосовых
Видео	sendVideo	Поддерживает MP4. Можно указать длительность, размеры, подпись
Документ	sendDocument	Любой файл. Можно добавить описание

```
#на python-telegram-bot
...
await update.message.reply_photo(
    photo = InputFile("photo.jpg"),
    caption = "Вот фото!")
```

```
#на aiogram
@router.message()
async def send_video(message:Mes...):
    video = FSInputFile("video.mp4")
    await message.answer_video(
        video = video,
        caption = "вот видео")
```

12. Интегрирование Telegram-bot с внешним REST API или базами данных. Осуществление запросов к внешним сервисам и обработка полученных данных.

Интеграция Telegram-бота с REST API — это когда бот не сам хранит данные, а обращается к внешнему сервису по HTTP, получает от него данные, обрабатывает их и отправляет пользователю в виде сообщения.

Пример:

```
import asyncio
import requests
from aiogram Bot, Dispatcher, types
from aiogram.filters import Command
TOKEN = "TOKEN"
bot = Bot(token=TOKEN)
dp = Dispatcher()
@dp.message(Command("currency"))
async def get_currency(message: types.Message):
    try:
        data = requests.get("https://www.cbr-xml-daily.ru/daily_json.js").json()
        usd = data["Valute"]["USD"]["Value"]
        await message.answer(f"Курс USD: {usd}")
    exceptException as e:
        await message.answer(f"ошибка {e}")
async def main():
    await dp.start_polling(bot)
if __name__ == "__main__":
    asyncio.run(main())
```

Делаем HTTP-запрос к внешнему REST API, которое возвращает JSON

→ Извлекаем из JSON нужную валюту.

13. Что такое web parsing и какие задачи он позволяет решать? Определение web scraping / parsing и их применение

Веб-скраппинг — автоматизированный процесс загрузки веб-страниц и извлечения с них информации (HTML, текст, ссылки, изображения) с целью последующей обработки или хранения.

Веб-парсинг — более узкое понятие, это процесс извлечения и обработки структурированных из веб-страниц, обычно из HTML-кода, для преобразования их в удобный формат.

Парсинг фокусируется на разборе кода, а скраппинг — на полном цикле: от запроса страницы до хранения.

Применение: мониторинг цен и ассортимента, поиск контактной информации, сбор информации, часто используется при отсутствии официального API.

14. Наиболее популярный модуль Python для отправки HTTP-запросов — requests и его преимущества.

Requests — наиболее популярный модуль Python для отправки HTTP-запросов. Модуль упрощает работу с методами API, умеет работать с сессиями, куки, JSON, широко используется для запросов к API, веб-скраппинга.

```
import requests
resp = requests.get("https://api.example.com/data", timeout=10)
resp.raise_for_status
data = resp.json()
print(data)
```

→ HTTP GET запрос

ограничиваем время ожидания ответа

resp.raise_for_status — в случае 4xx/5xx ошибок выбросит HTTPError

data = resp.json() — пробуем распарсить ответ как JSON и вернуть уже готовую структуру

15. Разница между использованием BeautifulSoup и lxml при парсинге HTML. Сравнение их по производительности, удобству и функциональности.

BeautifulSoup — максимально удобен, терпим к плохому HTML, имеет удобную навигацию по HTML дереву, но при этом медленный.

lxml — очень быстрый парсер на C с поддержкой XPath, отлично подходит для больших документов и сложных запросов, но сложнее для освоения

Лучше всего использовать их вместе для сохранения удобства и ускорения работы

Критерий	Производительность	Толерантность к ошибкам HTML	Удобство	XPath
BeautifulSoup	медленнее, подходит для небольших задач	Высокая, автоматически прощает ошибки HTML	Простой для освоения новичками, простые методы	Не поддерживает
lxml	быстрее, хорошо подходит для больших документов	Строже следует стандартам HTML, но умеет восстанавливать многие типичные ошибки	Более сложен для освоения	Поддержка XPath и XML-функций

XPath — формальный язык выражений, который позволяет описать путь к элементам внутри иерархической структуры документа и выбрать нужные узлы по имени, атрибутам, позиции или условиям.

Примеры: //div — все в документе

//a[@href] — все ссылки, у которых есть атрибут href

16. Пример кода для извлечения всех ссылок с веб-страницы с использованием bs4.

```
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin
url = "https://example.com"
html = requests.get(url, timeout=10).text
soup = BeautifulSoup(html, "html.parser")
links = []
for a in soup.find_all("a"):
    href = a.get("href")
    if not href:
        continue
    links.append(urljoin(url, href))
print(links)
```

- адрес страницы, с которой собираем ссылки
- GET запрос
- парсим HTML с BeautifulSoup
- перебираем все теги <a>
- получаем атрибут href
- пропускаем теги без href
- превращаем относительные ссылки в абсолютные.

17. Использование Selenium для обработки динамических веб-страниц. Особенности работа с динамическим контентом и примеры инициализации браузера.

Selenium управляет реальным браузером, что позволяет выполнить JS и подгрузить динамический контент. Также он может имитировать действия пользователя: выполнять клики, ввод текста, прокрутку.

Динамический контент требует явного ожидания загрузки страницы и последующего рендеринга. Для выполнения JS в реальном браузере с ожиданием полной отрисовки DOM хорошо подходит Selenium.

Пример инициализации Chrome

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

opts = Options() - создаем объект опций для Chrome
opts.add_argument("--headless=new") - режим без окна
opts.add_argument("user-agent=MyScraper/1.0") - указываем User-agent для страницы
driver = webdriver.Chrome(options=opts) - создаем экземпляр драйвера с указанными опциями
driver.get("https://example.com") - загружаем указанный URL в управляемом браузере.
```

18. XPath и его использование для поиска элементов на странице. Синтаксис XPath и разница с CSS-селекторами.

XPath - формальный язык выражений, который позволяет описать путь к элементам внутри иерархической структуры документа и выбрать нужные узлы по имени, атрибутам, позиции или условиям.

Примеры:
```
//div - все <div> в документе
//a[@href] - все ссылки, имеющие href
//button[text()="ok"] - кнопка по тексту
```

```
from selenium.webdriver.common.by import By
driver.find_element(By.XPATH, "//button[@type='submit']")
```

XPath мощнее для сложной навигации по дереву, когда нужны точные условия.
CSS-селекторы проще и быстрее для большинства задач

19. Организация ожидания загрузки элементов в Selenium. Методы "явного" и "неявного" ожидания и их применение

В Selenium организация ожидания загрузки элементов критически важна для стабильной работы с динамическим контентом

Неявное ожидание - все find_element будут ждать до таймаута, перед выбрасыванием исключения

```
...
driver.implicity_wait(10)
el = driver.find_element(By.ID, "dynamic")
```

Подходит для простых страниц, где элементы быстро появляются

Явное ожидание - ожидает конкретное условие

```
...
wait = WebDriverWait(driver, 15)
el = wait.until(EC.visibility_of_element_located((
        By.CSS_SELECTOR, "div.content")))
```

Позволяет ожидать нужного состояния элемента

20. Проблемы, которые могут возникнуть при разработке парсеров для сайтов с защитой от ботов. Типичные меры защиты и возможные способы обхода (с учетом законности и этичности)

Основные проблемы: ограничение кол-ва запросов с IP, блокировка бота по User-Agent, капчи, анализ поведения, динамическая прогрузка контента.
Способы обхода: Использование официального API, получение официального разрешения владельца ресурса, снижение нагрузки на сайт, явная авторизация как бота.
Серые темки для обхода: использование прокси, VPN, подключение решателя капчи, маскировка под человека.

21. Как можно предотвратить блокировку IP при интенсивном парсинге веб-страниц? Использование прокси-серверов и изменение user-agent

Предотвращение блокировок по IP при интенсивном парсинге требует имитации человеческого поведения и распределения нагрузки. Основные методы: ротация прокси, изменение User-agent, контроль частоты запросов, использование разных сессий и кук

22. Отличия CSS селекторов от XPath в контексте поиска элементов на странице. Сравнение удобства и возможностей обоих подходов.

Вопрос. 18

CSS: проще для классов / структур. Или быстрее писать, широко используется во фронтенде
XPath: мощнее и гибче для сложной навигации по DOM, поиска по тексту и узлам

Критерий	Синтаксис	Навигация	Поиск по тексту	Производительность	Читаемость
XPath	Более громоздкий	Вверх и вбок по осям	Полноценные ф-ции text() и contains()	Иногда медленнее	Сложнее при больших выражениях
CSS-селектор	Короткий, похож на CSS	Вниз по дереву	Ограничен	Обычно быстрее	Более человекочит.

23. Использование регулярных выражений для извлечения данных из текстового блока. Пример задачи и решения с применением модуля re

Регулярные выражения — удобный инструмент для извлечения структурированных фрагментов из текста.

Пример:

```
import re
text = "Новости: 17.01.2026, 19.01.2026"
dates = re.findall(r'\d{2}\.\d{2}\.d{4}', text)
print(dates) → '17.01.2026', '19.01.2026'
```

возвращает все значения

\d — любая цифра (0-9)
{2}, {4} — точное кол-во совпадений
\. — точка

24. AJAX и его влияние на процесс парсинга веб-страниц. Усложнение сбора данных AJAX-запросами и работа с ними

AJAX — способ подгружать данные асинхронно через JS, без перезагрузки страницы. Для парсинга это значит, что нужный контент может отсутствовать в исходном HTML и требовать прогрузки. Данные часто в JSON, а не в HTML, что дает возможность вызывать тот же API напрямую и получить структурированный ответ.

Способы работы с AJAX:

- Найти сетевой запрос в DevTools и вызвать API endpoint напрямую
- Использовать Selenium / Playwright и прогружать элементы

25. Создание базового REST API приложения с использованием FastAPI. Пример минимального кода и объяснение его структуры

Создание приложения на FastAPI:

1) создать экземпляр приложения app = FastAPI()
2) описываем маршруты с декораторами @app.get / @app.post
3) запуск через uvicorn main:app --reload

```
from fastapi import FastAPI
app = FastAPI()
```

@app.get("/") — декоратор, который регистрирует HTTP GET маршрут по пути /

```
def read_root():
    return {"message": "Hello, World"}
```

— возвращаемый словарь, который автоматически сериализуется в JSON и отправляется клиенту с кодом 200

26. Особенности асинхронных обработчиков в FastAPI по сравнению с синхронными. Почему FastAPI оптимизирован под async и как это влияет на производительность.

Асинхронные обработчики в FastAPI позволяют эффективно обрабатывать множество одновременных запросов без блокировки основного цикла событий. Синхронный обработчик выполняется как обычный синхронный код (выполнение всех операций функций с блокировкой потока, в котором запущено).

FastAPI рассчитан на ASGI-сервера (Uvicorn), которые поддерживают неблокирующую обработку запросов. Так как веб-сервисы чаще всего ждут ответа от внешних ресурсов (БД, API, т.д.), намного эффективнее вместо блокировки потока и ожидания ответа выполнять другие задачи, а дождавшись ответа вернуться к нему.

27. Настройка маршрутизации (routing) в FastAPI. Принцип работы маршрутов, использование path- и query-параметров.

Маршрутизация в FastAPI задаётся декораторами @app.get / post / ... над функциями-обработчиками.

Path-параметры вшиваются в путь в { } — часть URL и являются обязательными

```
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    return {"user_id": user_id}
```

/users/123 → user_id = 123

Query-параметр — идет после ? в строке запроса и используется для фильтров, пагинации и опциональных настроек

```
@app.get("/users/")
async def get_users(limit: int = 10, skip: int = 0, q: str = None):
    return {"limit": limit, "skip": skip, "q": q}
```

/users/ → limit = 10, skip = 0, q = None

/users/?limit=5&skip=10&q=John $\xrightarrow{q = None}$ limit = 5, skip = 10, q = "John"

28 Использование Pydantic для валидации данных в FastAPI. Пример описания модели данных и валидации входящих запросов.

Pydantic в FastAPI описывает схемы данных как Python-классы, автоматически валидирует и преобразует входящий JSON в объекты с типами, а при ошибках возвращает понятные HTTP-ошибки. Все это делает валидацию явной, безопасной и документируемой.

Pydantic использует BaseModel для описания структуры данных (поля, типы, значения по умолчанию) и выполняет валидацию при создании модели — FastAPI применяет это автоматически к телу запроса и ответам.

```
from pydantic import BaseModel
from typing import Optional

class User (BaseModel):  — Определение схемы данных
    id: int
    name: str
    age: int = 18

@app.post ("/users/")
async def create_user (user: User):  — обработчик запроса; параметр user: User говорит FastAPI
    return user                       взять тело запроса JSON, распарсить и валидировать
                                      его как User, затем передать в функцию
```

29. Генерация документации для API с FastAPI. Интеграция Swagger UI и ReDoc, а также их возможности по настройке.

FastAPI автоматически генерирует документацию API на основе схемы OpenAPI на основе маршрутов, типов и Pydantic-моделей. Она доступна в двух интерфейсах — Swagger UI (/docs) и ReDoc (/redoc). Оба инструмента можно гибко настраивать: менять цвета, логотипы, параметры отображения, подключать собственные CSS/JS и даже полностью переопределять HTML-страницы.

```
app = FastAPI (redoc_url = "/api-docs" | = None) | (docs_url = "swagger" | = None)
```

30. Обработка исключений в FastAPI. Создание обработчиков ошибок (exception handlers) и их регистрации.

В FastAPI обработка ошибок реализована через встроенный механизм exception handlers, что позволяет централизовано управлять тем, как приложение реагирует на различные исключения. Можно обрабатывать как стандартные исключения (HTTPException), так и свои собственные (exception_handler)

```
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse
class MyException (Exception):
    def __init__ (self, name: str):
        self.name = name

app. = FastAPI()
@app.exception_handler (MyException)
async def custom_exception_handler (request: Request,
                                    exc: MyException):
    return JSONResponse (status_code = 418,
                         content = {"message": "Произошла ошибка"})
@app.get ("custom_error")
async def custom_error():
    raise MyException (name = "что-то пошло не так")
```

```
@app.get ("/items/{item_id}")
async def read_item (item_id: int):
    if item_id == 0:
        raise HTTPException (status_code = 404,
                             detail = "Item not found")
    return {"item_id": item_id}
```

31. Background Tasks в FastAPI и их применение. Механизм выполнения фоновых задач и примеры

Background Tasks – механизм выполнения задач в фоновом режиме после отправки HTTP-ответа клиенту. Это позволяет не блокировать основной поток обработки запросов, делая API более отзывчивым. Задачи выполняются в том же процессе, что и приложение, поэтому они не предназначены для тяжелых операций.

```
from fastapi import FastAPI, BackgroundTasks
app = FastAPI()

def write_log(msg: str):
    with open("log.txt", "a") as f
        f.write(msg + "\n")

@app.post("/notify/")
async def notify(email: str, tasks: BackgroundTasks):
    tasks.add_task(write_log, f"Sent to {email}") – добавили задачу в BackgroundTasks
    return {"status": "ok"}
```

32. Пример интеграции FastAPI с внешним источником данных

FastAPI легко интегрируется с внешними источниками через HTTP-запросы через клиентские библиотеки (requests, httpx).

```
from fastapi import FastAPI
import requests
API_KEY = "key"
BASE_URL = "https://api.openweathermap.org/data/2.5/weather"

@app.get("/weather/{city}")
def get_weather(city: str):
    params = {"q": city, "appid": API_KEY, "units": "metric", "lang": ru}
    response = requests.get(BASE_URL, params=params)
    if response.status_code == 200:
        data = response.json()
        return {"город": data["name"],
                "температура": data["main"]["temp"]}
    return {"error": "Не удалось получить данные"}
```

33. Разница между методами POST и PUT, их использование в FastAPI. Особенности каждого метода в контексте создания и обновления ресурсов.

• POST используется для создания новых ресурсов, когда клиент не знает точный URL. Каждый повторный вызов может создать дубликаты. Используется для: добавления записи в БД, запуска процесса, отправки формы.

```
...
@app.post("/users/")
def create_user(...)
    ...
    return {"message": "Пользователь создан"}
```

```
...
@app.put("/users/{user_id}")
def update_user(user_id: int, ...):
    ...
    return {"message": "Польз-ль обновлен"}
```

• PUT применяется для полного обновления или создания ресурса по полному URL. Каждый повторный вызов дает тот же результат без дублирования.

28 Использование Pydantic для валидации данных в FastAPI. Пример описания модели данных и валидации входящих запросов.

Pydantic в FastAPI описывает схемы данных как Python-классы, автоматически валидирует и преобразует входящий JSON в объекты с типами, а при ошибках возвращает понятные HTTP-ошибки. Все это делает валидацию явной, безопасной и документируемой.

Pydantic использует BaseModel для описания структуры данных (поля, типы, значения по умолчанию) и выполняет валидацию при создании модели — FastAPI применяет это автоматически к телу запроса и ответам.

```
from pydantic import BaseModel
from typing import Optional

Class User (BaseModel):  — Определение схемы данных
    id: int
    name: str
    age: int = 18

@app.post ("/users/")
async def create_user (user: User):  — обработчик запроса; параметр user: User говорит FastAPI
    return user                       взять тело запроса JSON, распарсить и валидировать
                                      его как User, затем передать в функцию
```

29. Генерация документации для API с FastAPI. Интеграция Swagger UI и ReDoc, а также их возможности по настройке.

FastAPI автоматически генерирует документацию API на основе схемы OpenAPI на основе маршрутов, типов и Pydantic-моделей. Она доступна в двух интерфейсах — Swagger UI (/docs) и ReDoc (/redoc). Оба инструмента можно гибко настраивать: менять цвета, логотипы, параметры отображения, подключать собственные CSS/JS и даже полностью переопределять HTML-страницы.

```
app = FastAPI (redoc_url = "/api-docs" | = None) | (docs_url = "swagger" | = None)
```

30. Обработка исключений в FastAPI. Создание обработчиков ошибок (exception handlers) и их регистрации.

В FastAPI обработка ошибок реализована через встроенный механизм exception handlers, что позволяет централизовано управлять тем, как приложение реагирует на различные исключения. Можно обрабатывать как стандартные исключения (HTTPException), так и свои собственные (exception_handler)

```
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse
Class MyException (Exception):
    def __init__ (self, name: str):
        self.name = name

app. = FastAPI()
@app.exception_handler (MyException)
async def custom_exception_handler (request: Request,
                                    exc: MyException):
    return JSONResponse (status_code = 418,
                         content = {"message": "Произошла ошибка"})
@app.get ("custom_error")
async def custom_error():
    raise MyException (name = "что-то пошло не так")

@app.get ("/items/{item_id}")
async def read_item (item_id: int):
    if item_id == 0:
        raise HTTPException (status_code = 404,
                             detail = "Item not found")
    return {"item_id": item_id}
```

31. Background Tasks в FastAPI и их применение. Механизм выполнения фоновых задач и примеры

Background Tasks – механизм выполнения задач в фоновом режиме после отправки HTTP-ответа клиенту. Это позволяет не блокировать основной поток обработки запросов, делая API более отзывчивым. Задачи выполняются в том же процессе, что и приложение, поэтому они не предназначены для тяжелых операций.

```
from fastapi import FastAPI, BackgroundTasks
app = FastAPI()

def write_log(msg: str):
    with open("log.txt", "a") as f
        f.write(msg + "\n")

@app.post("/notify/")
async def notify(email: str, tasks: BackgroundTasks):
    tasks.add_task(write_log, f"Sent to {email}") – добавили задачу в BackgroundTasks
    return {"status": "ok"}
```

32. Пример интеграции FastAPI с внешним источником данных

FastAPI легко интегрируется с внешними источниками через HTTP-запросы через клиентские библиотеки (requests, httpx).

```
from fastapi import FastAPI
import requests
API_KEY = "key"
BASE_URL = "https://api.openweathermap.org/data/2.5/weather"

@app.get("/weather/{city}")
def get_weather(city: str):
    params = {"q": city, "appid": API_KEY, "units": "metric", "lang": "ru"}
    response = requests.get(BASE_URL, params=params)
    if response.status_code == 200:
        data = response.json()
        return {"город": data["name"],
                "температура": data["main"]["temp"]}
    return {"error": "Не удалось получить данные"}
```

33. Разница между методами POST и PUT, их использование в FastAPI. Особенности каждого метода в контексте создания и обновления ресурсов.

POST используется для создания новых ресурсов, когда клиент не знает точный URL. Каждый повторный вызов может создать дубликаты. Используется для: добавления записи в БД, запуска процесса, отправки формы.

```
...
@app.post("/users/")
def create_user(...)
    ...
    return {"message": "Пользователь создан"}
```

```
...
@app.put("/users/{user_id}")
def update_user(user_id: int, ...):
    ...
    return {"message": "Польз-ль обновлен"}
```

PUT применяется для полного обновления или создания ресурса по полному URL. Каждый повторный вызов даёт тот же результат без дублирования.

34. Создание главного окна приложения с помощью Tkinter. Основные шаги создания окна и инициализации основного цикла.

Tkinter — стандартная библиотека Python для создания графических интерфейсов.

Шаги создания окна:

1) Импорт библиотеки tkinter
2) Создание экземпляра главного окна
3) Настройка окна
4) Добавление виджетов
5) Запуск основного цикла

#1 import tkinter as tk

#2 root = tk.Tk() — Tk() - инициализирует главное окно

#3 { root.title ("Приложение") — название
root.geometry ("400x500") - ширина x высота

#4 { button = tk.Button (root, text = "Закрыть", command = = root.destroy)
button.pack

#5 root.mainloop()

Метод mainloop() запускает бесконечный цикл обработки событий tkinter: отслеживание движения мыши, нажатия клавиш и т.д.

35 Способы размещения виджетов в Tkinter (pack, grid, place). Особенности каждого метода и примеры их применения.

Метод pack()	Метод grid()	Метод place()
Размещает виджеты блоками относительно друг друга. • Автоматически подстраивает размеры контейнера под содержимое. • Удобен для простых интерфейсов, где не нужна точная сетка tk.Label (...).pack (side="top"/side="left"/"right"/"bottom"	• Размещает виджеты в таблице (строки и столбцы) • Позволяет объединять ячейки • Идеален для форм, таблиц tk.Label (root, text="Имя:").grid(row=0, column=0) tk.Entry (root).grid(row=0, column=1)	• Абсолютное позиционирование по координатам (x, y). • Можно задавать размеры и относительные позиции (relx, rely) • Подходит для точного контроля расположения, но сложнее поддерж. адаптивность. tk.Label (root, text = "Label"), place (x=50, y=100) *x, y - координаты в пикселях отн. верхнего левого угла контейнера

36. Добавление кнопки (Button) на форму и связывание ее с функцией обработчиком события. Пример кода с созданием кнопки и назначением команды

В Tkinter кнопка создается с помощью виджета Button, а обработчик события назначается через параметр command.

Пример:

```
import tkinter as tk
def on_click():
    print ("click")
root = tk.Tk()
btn = tk.Button (root, text = "Нажми", command = on_click)
btn.pack ()
root.mainloop()
```

37. Создание иерархии меню в Tkinter. Создание главного меню, выпадающих списков и подключения команд к пунктам меню.

В Tkinter меню создается с помощью виджета Menu. Оно может содержать пункты главного меню, выпадающие списки и команды, привязанные к функциям-обработчикам

Пример кода:

```
import tkinter as tk

root = tk.Tk()
menubar = tk.Menu(root)  - создаем главное меню и привязываем его к окну
root.config(menu=menubar)  - подключаем меню к окну
second_menu = tk.Menu(menubar, tearoff=0)  - создаем подменю. tearoff=0 - убирает возможность отрыва меню в отд. окно
second_menu.add_command(label="Выход",
                        command=root.destroy)  - добавляем пункт в подменю

menubar.add_cascade(label="Доп. меню",
                    menu=second_menu)  - добавляем подменю в меню

root.mainloop()
```

38. Обновление текста или свойств виджета Label в Tkinter. Использование методов для изменения отображаемых данных.

В Tkinter виджет Label используется для отображения текста или изображений. Его свойства можно менять динамически — например, обновлять текст при нажатии кнопки или изменять цвет.

Основные методы:

1) config() / configure()

Позволяет изменить свойства виджета после его создания

```
label.config(text="Новый текст", bg="yellow")
```

2) ["property"]

Можно обращаться к свойствам как к ключам

```
label["text"] = "Обновленный текст"
label["fg"] = "blue"
```

Свойства Label для изменения

- text — текст
- fg — цвет текста
- bg — цвет фона
- font — шрифт и размер
- image — картинка вместо текста

39. Canvas и его использование для рисования графических элементов. Возможности Canvas и примеры рисования фигур

Canvas — специальный виджет Tkinter, предназначенный для рисования графических элементов: линий, фигур, текста, изображений. Он дает возможность создавать простую графику прямо в окне приложения.

Возможности Canvas:

- Рисование линий, прямоугольников, овалов, многоугольников
- Добавление текста и изображений
- Перемещение, изменение и удаление элементов
- Обработка событий (клики по нарисованным объектам)

Основные методы рисования:

```
create_line(x1, y1, x2, y2, ...) - линия
-||- rectangle(x1, y1, x2, y2, ...) - прямоугольник
-||- oval(x1, y1, x2, y2, ...) - овал
-||- polygon(x1, y1, x2, y2, ...) - многоугольник
-||- text(x, y, text="...") - текст
-||- image(x, y, image=...) - изображение
```

40. Реализация ввода данных пользователем через виджет Entry. Метод получения значения из текстового поля и его последующей обработки.

Entry – основной виджет для ввода текста пользователем.

```
entry = tk.Entry (root)  - Создание поля ввода в окне root
entry.pack()
```

Для получения введенного значения используется:

value = entry.get(), | entry.get()[индексы]

Он возвращает строку, которую можно обрабатывать дальше

```
entry.delete (0, tk.END)   – удаляет текст в указанном диапазоне
entry.insert (index, text)  – вставляет текст в указанную позицию
entry.config (show = "*")  – заменяет отображаемые символы на указанный знак
```

41. Использование Listbox для отображения списка элементов и обработки выбора пользователя. Пример создания Listbox и работы с его элементами.

Listbox – виджет, который отображает список строк, где пользователь может:

- выбирать элементы
- прокручивать список
- реагировать на выбор

Пример:

```
import tkinter as tk
root = tk.Tk()
lb= tk.listbox (root)   } создаем Listbox в root
lb.pack()
for x in ["Раз","Два","Три"]:
    lb.insert (tk.END, x)        – добавляем элементы в список
def on_select (event):  - обработчик выбора элемента из списка
    index = lb.curselection()  – получаем индекс выбранного элемента
    if index:
        value = lb.get(index))  – получаем текст элемента
lb.bind ("<<ListboxSelect>>", on_select)  – привязываем обработчик на выбор из списка
root.mainloop()
```

42. Создание диалогового окна для отображения ошибок или информационных сообщений в Tkinter. Использование таких модулей, как messagebox, для создания окна сообщений

В Tkinter для отображения ошибок, предупреждений и информационных сообщений используется модуль tkinter.messagebox. Он позволяет быстро создавать стандартные диалоговые окна.

Готовые функции:

```
messagebox.showinfo() – информационное
  -||-      .showwarning() – предупреждение
  -||-      .showerror() – ошибка
  -||-      .askyesno() – вопрос (да/нет)
  -||-      .askokcancel() – вопрос (ок/отмена)
  -||-      .askyesnocancel() – вопрос (да/нет/отмена)
```

```
import tkinter as tk
from tkinter import messagebox
def show_info():
    messagebox.showinfo ("Информация",
                "Операция выполнена")
root = tk.Tk()
tk.Button (root, text="Клик", command =
                show_info().pack()
root.mainloop()
```

43. Настройка обработки событий клавиатуры в Tkinter. Примеры связывания событий нажатия клавиш с функциями

В Tkinter клавиатурные события обрабатываются через метод bind, который привязывает нажатие клавиши к функции обработчику

widget.bind("<KeyPress>", handler) — функция обработчик

куда привязываем — какую кнопку обрабатываем

<KeyPress> — нажатие любой клавиши
<KeyRelease> — отпускание клавиши
<Control-s> — комбинация Ctrl+s

Обработчик получает event. Полезные свойства event:
- char — введенный символ
- keysym — имя клавиши
- keycode — числовой код клавиши
- x, y — координаты мыши

44. Особенности разработки многооконных приложений в Tkinter. Использование дополнительных окон (Toplevel) и их управление.

В многооконных приложениях Tkinter главное окно создается от Tk, а все дополнительные — от специального класса Toplevel, который является отдельным окном, подвязанным к корневому.

Основные особенности разработки многооконных приложений:
- Главное окно всегда создается только через Tk(), все остальные — Toplevel()
- Каждое окно живет своей жизнью: собственные события, виджеты, методы управления
- Можно управлять поведением окна: запретить изменение размеров, задать размеры, закрепить окно поверх всех

Пример:
```
import tkinter as tk
def open_window():
    win = tk.Toplevel(root)
    win.title("Новое окно")
    win.geometry("250x150")
    tk.Label(win, text="Второе окно").pack()
    tk.Button(win, text="Закрыть", command=win.destroy).pack()
root = tk.Tk()
root.title("Главное окно")
tk.Button(root, text="Открыть окно", command=open_window).pack()
root.mainloop()
```

*. wait_window() — ждёт закрытия окна
- focus_set() — переводит фокус
- grab_set() — блокирует другие окна

Окна могут обмениваться данными через:
- Передачу данных как параметр функции
- использование общих переменных
- использование callback функций для передачи из дочернего окна в главное
- использование классов.

45. Интегрирование Tkinter с задачами, выполняемыми в отдельных потоках (multithreading). Особенности работы GUI в многопоточных приложениях и способы безопасного обновления интерфейса

В Tkinter GUI работает только в главном потоке. Все обновления интерфейса из других потоков запрещены, это может вызвать зависания, вылеты или непредсказуемое поведение.

Способы безопасного обновления GUI

1) Использовать after() для периодической проверки данных

Принцип: • Фоновый поток выполняет тяжелую работу
• Результаты складываются в очередь (queue.Queue)
• Главный поток каждые N миллисекунд проверяет очередь через after() и обновляет интерфейс

```
import tkinter as tk, threading, queue, time
def worker(q):
    for i in range(5):
        time.sleep(1)
        q.put(i)
def update():
    if not q.empty():  - если в очереди есть данные
        label.config(text=q.get())  - обновляем текст элементом из очереди
    root.after(100, update)  - планируем вызов этой же функции через 100 мсек
root = tk.Tk()
label = tk.Label(root, text="Ожидание...")
label.pack()
q = queue.Queue()  - создаем очередь для обмена данными между потоками и GUI
threading.Thread(target=worker, args=(q,), daemon=True).start()  - запускаем фоновый поток (выполняемая ф-ция, передаем в нее очередь q, автоматически завершаем при закрытии программы)
update()  - запускаем цикл update()
root.mainloop()
```

2) Использование after() как таймер для фоновой задачи

```
def long_task(step=0):
    if step<10:
        label.config(text=f"Шаг {step}")
        root.after(500, long_task, step+1)
long_task()
```

3) Использование threading + безопасный вызов GUI через after().

Фоновый поток вызывает GUI не напрямую, а через планирование

```
def worker():
    time.sleep(2)
    root.after(0, lambda: label.config(text="Изменен"))
```

46. Метод mainloop() в Tkinter и почему он является необходимым элементом приложения. Его роль в поддержании жизненного цикла окна.

mainloop() – главный цикл обработки событий. Он запускает внутренний механизм Tkinter, который:

- отслеживает действия пользователя
- обновляет интерфейс
- реагирует на события
- перерисовывает окна
- поддерживает приложение живым

Без mainloop() окно откроется и сразу же закроется, т.к. программа завершится.

Роль mainloop() в жизненном цикле окна: 1) Создание постоянного цикла обработки событий
2) Не дает программе завершиться
3) Обеспечивает обновление интерфейса
4) Обрабатывает закрытие окна

47. Использование модуля ttk для создания современных и стилизованных виджетов в Tkinter. Преимущества ttk и для каких целей его целесообразно использовать

Модуль ttk – это современное расширение Tkinter, которое добавляет стильные, аккуратные виджеты, делая интерфейс визуально более приятным

Преимущества ttk: 1) Современный внешний вид
2) Возможность менять тему всего приложения одной строкой
3) Возможность создавать и применять собственные стили
4) Новые виджеты (Treeview, Notebook, Combobox, Progressbar и др.)

```
import tkinter as tk
from tkinter import ttk
root = tk.Tk()
style = ttk.Style()
style.theme_use("clam")
btn = ttk.Button(root, text="OK")
btn.pack()
root.mainloop()
```

Использовать ttk стоит когда:
- нужно улучшить оформление без сложных библиотек
- требуется единообразие интерфейса на разных ОС
- требуется гибкая стилизация

58. Описание архитектуры приложения, в котором интегрированы Telegram-bot, парсинг/анализ данных, FastAPI и Tkinter. Задачи, которые оно может решать. Распределение функциональности между компонентами и обеспечение взаимодействия между ними

Tkinter – в качестве локального интерфейса. Функционал:
- Панель управления для администратора
- Загрузка данных, просмотр результатов
- Отправление запросов в FastAPI.

FastAPI – центральный сервер:
- Единая точка логики
- Принимает запросы от Tkinter и Telegram-бота
- Запуск функций анализа/сбора данных
- Возвращает результаты в JSON или файлах

Модуль анализа данных:
- Выполнение тяжелых вычислений
- Возвращает результаты FastAPI

Telegram-бот:
- Принимает команды и файлы
- Передает все в FastAPI
- Получает результаты и отправляет пользователю

Взаимодействие: Tkinter → FastAPI, Telegram-бот → FastAPI } отправка запросов для анализа и получение результатов
FastAPI → Модуль анализа данных — вызов функций обработки данных

Возможности такого приложения: Предоставление пользователям анализировать предоставляемые данные как через локальный интерфейс, так и через Telegram-бота удаленно.